

ELEN6350 VLSI Design Lab

Stellavis NBU
N-Body Accelerator
Datasheet (V1.0)

Charlotte Chen (hc3558)

Lucy He (lh3365)

Ruomeng Ma (rm4305)

Chenxi Shen (cs4634)

February 2026



Stellavis NBU 2026

v1.0

1 Overview

- Operation Frequency: 40 MHz
- Maximum Clock Frequency: 100 MHz
- 16-bit 1 MHz Parallel IO Channels
- 4 Parallel Processing Elements
- Maximum of 256 Bodies
- Reconfigurable Physics Engine
- Processing Speed: 4M pairs of interactive force calculation per second

2 General Description

The Stellavis NBU is a reconfigurable N-body compute accelerator that evaluates pairwise interactions and accumulates per-body force vectors for particle simulation workloads.

The Stellavis NBU uses a programmable lookup-table force engine that enables a variety of distance-dependent interaction laws without re-designing the compute datapath. The NBU accepts streamed body state data and configuration parameters, performs high-throughput interaction accumulation across bodies, and outputs force results for downstream integration by an external processor. An FPGA-compatible serial interface provides control and status, including loading force tables, setting numerical parameters, and starting or pausing computation.

3 Functional Block Diagram

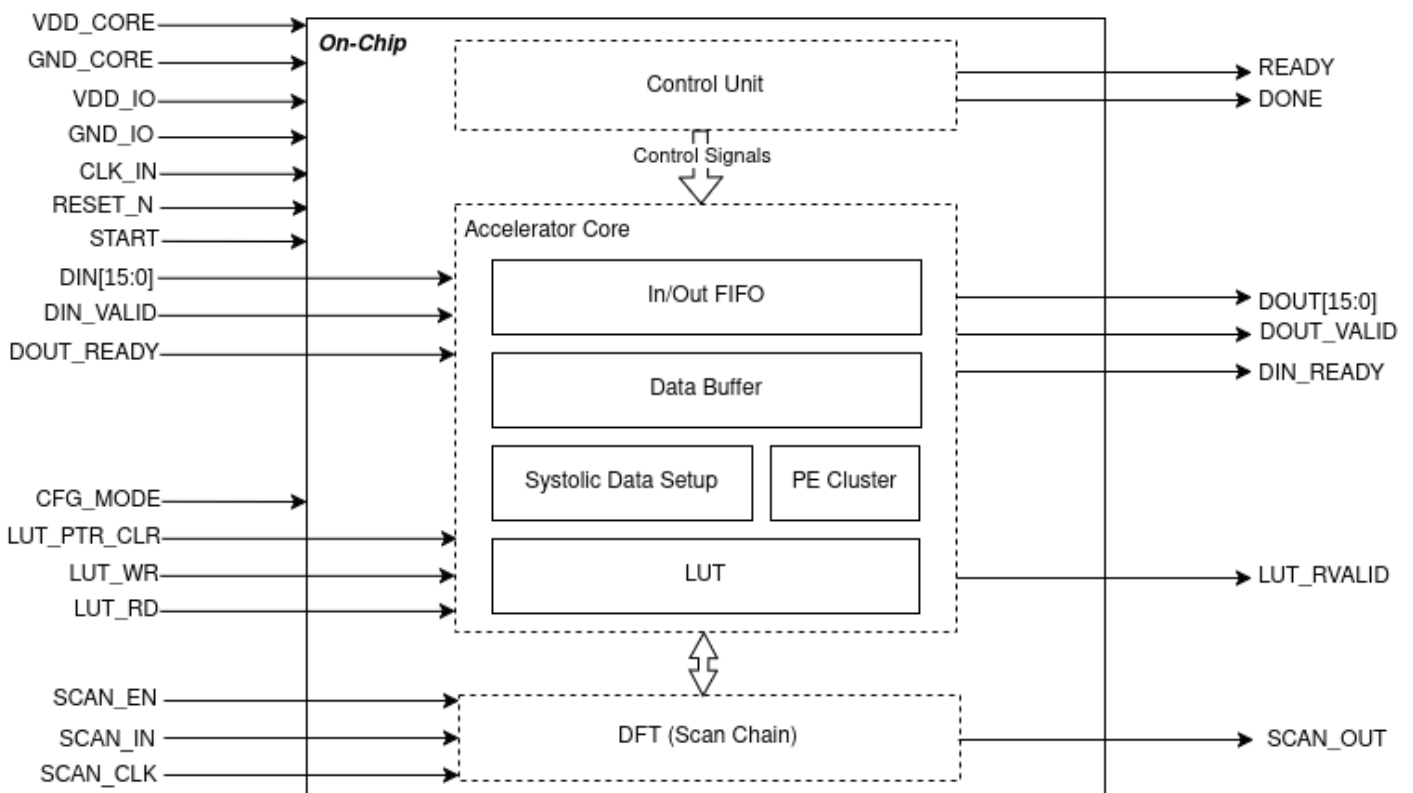


Figure 1: Chip Top-level Diagram

4 Table of Contents

1	Overview	1
2	General Description	1
3	Functional Block Diagram	1
4	Table of Contents	2
5	Technical specification	3
6	Pinout	3
7	Theory of Operation	5
7.1	NBU Core	5
7.1.1	FIFO	5
7.1.2	Data Buffer	5
7.1.3	Systolic Processing Element (PE) Array	5
7.1.4	Lookup Table Memory	6
7.2	Finite State Machine (FSM)	6
7.2.1	Top Level FSM	7
7.2.2	Configure-LUT FSM	8
7.3	DFT	9
8	Application Information	9
8.1	FPGA-Based System Integration	9
8.1.1	Parallel Data Interface	9
8.1.2	Control and Mode Selection	9
8.1.3	Lookup-Table Configuration Protocol	10
8.1.4	Compute Operation Protocol	10
8.2	Functional Applications and Supported Physics Laws	10
8.2.1	Newtonian Gravity	11
8.2.2	Coulomb Electrostatic Interaction	11
8.2.3	Lennard–Jones 12–6 Interaction	11
8.2.4	Linear Spring Interaction	11
8.2.5	Application Flexibility	11

5 Technical specification

Parameter	Specification
Technology	TSMC 65nm
Die Size	1mm x 1mm
Operating Frequency	40MHz
Maximum Clock Frequency	100MHz
Supply Voltage	1.0V (Core), 2.5 V (I/O)
IO Rate	1MHz
Power Consumption	< 50 mW

6 Pinout

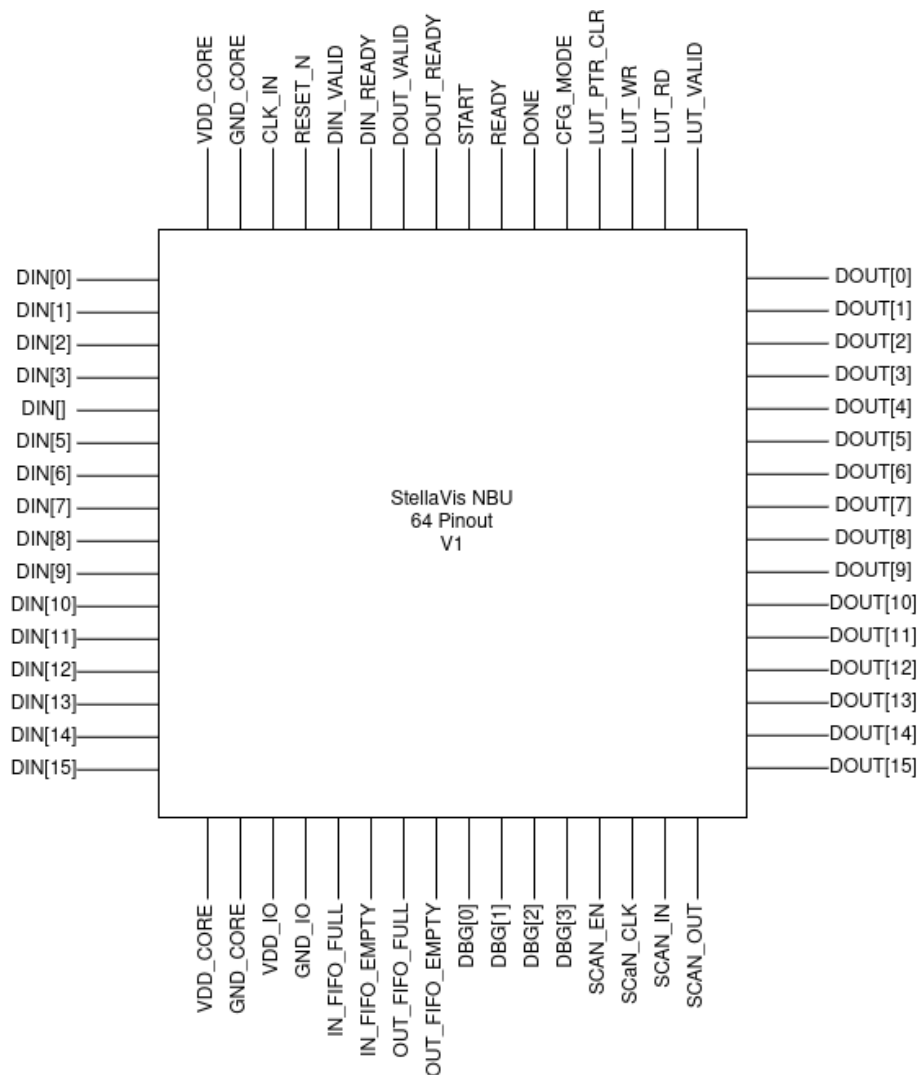


Figure 2: Pinout diagram of Stellavis NBU

Pin Configuration and Description (64 Pins Total)			
Pin Number	I/O	Pin Name	Description
Power and Ground Pins			
1–2	Power	VDD_CORE	Core supply voltage pins for internal logic and SRAM.
3–4	Power	GND_CORE	Ground reference pins for core logic.

5	Power	VDD_IO	I/O supply voltage for external interfaces.
6	Power	GND_IO	Ground reference for I/O domain.
Clock and Reset			
7	Input	CLK_IN	Global system clock driving the accelerator.
8	Input	RESET_N	Active-low synchronous reset.
Data Interface			
9–24	Input	DIN[15:0]	Parallel input data bus. Used for particle data in compute mode and LUT data during configuration.
25	Input	DIN_VALID	Indicates that DIN[15:0] is valid.
26	Output	DIN_READY	Asserted when the accelerator can accept input data.
27–42	Output	DOUT[15:0]	Parallel output data bus for computed force results or LUT readback data.
43	Output	DOUT_VALID	Indicates that DOUT[15:0] is valid.
44	Input	DOUT_READY	Indicates that the external system is ready to receive output data.
Control and Status Signals			
45	Input	START	Starts LUT configuration or computation depending on CFG_MODE.
46	Output	READY	Indicates that the accelerator is idle and ready for a new command.
47	Output	DONE	Asserted when the current operation has completed.
LUT Interface			
48	Input	CFG_MODE	Configuration mode select.
49	Input	LUT_PTR_CLR	Clears the internal LUT address pointer to entry 0.
50	Input	LUT_WR	Sequential LUT write strobe. Writes DIN[15:0] and auto-increments the LUT pointer.
51	Input	LUT_RD	Sequential LUT read strobe.
52	Output	LUT_RVALID	Indicates valid LUT read data on DOUT[15:0].
FIFO Status and Debug			
53	Output	IN_FIFO_FULL	Asserted when the input FIFO is full.
54	Output	IN_FIFO_EMPTY	Asserted when the input FIFO is empty.
55	Output	OUT_FIFO_FULL	Asserted when the output FIFO is full.
56	Output	OUT_FIFO_EMPTY	Asserted when the output FIFO is empty.
General Debug Signals			
57–60	Output	DBG[3:0]	General-purpose debug outputs for observing internal FSM or pipeline state.
Scan Chain			
61	Input	SCAN_EN	Enables scan mode for design-for-test.
62	Input	SCAN_CLK	Scan clock input for shifting scan chain data.
63	Input	SCAN_IN	Serial scan data input.
64	Output	SCAN_OUT	Serial scan data output.

Table 1: Stellavis NBU pin configuration and description.

7 Theory of Operation

7.1 NBU Core

The accelerator core performs high-throughput evaluation of pairwise interactions and accumulation of per-body acceleration vectors. It operates on streamed body state data and a programmable distance-dependent interaction function. The core consists of clock-domain synchronization FIFOs, data buffers, a systolic processing element array, and replicated lookup-table memory blocks.

7.1.1 FIFO

Input and output FIFOs provide safe clock-domain crossing between the host interface clock and the internal accelerator clock. These FIFOs decouple communication timing from computation, allowing the accelerator to operate at an optimized internal frequency while maintaining reliable data transfer. FIFO status signals are monitored by the FSM to prevent overflow and underflow conditions.

7.1.2 Data Buffer

Internal data buffers store streamed body parameters and intermediate results during computation. The buffers enable efficient reuse of body data across multiple interaction evaluations and support continuous streaming of input and output data without stalling the systolic array. Buffer depth and access scheduling are managed to sustain full throughput of the processing elements.

7.1.3 Systolic Processing Element (PE) Array

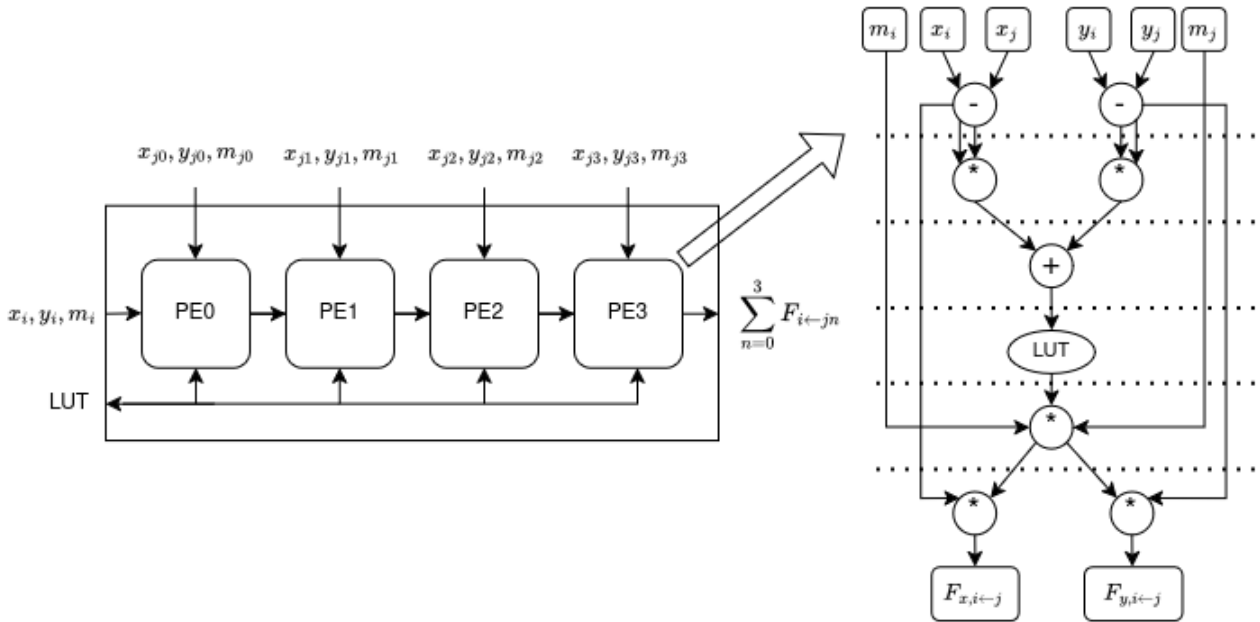


Figure 3: Four-PE systolic array architecture and per-PE computation flow. Each processing element computes relative displacement, evaluates the programmable $g(r^2)$ function using a local LUT, and accumulates a vector interaction contribution.

The Stellavis NBU accelerator core is implemented as a one-dimensional systolic array of four processing elements (PEs). The systolic organization enables continuous streaming of source body data through the array while accumulating interaction contributions for a target body in a deeply pipelined manner. After pipeline fill, the array sustains one pairwise interaction per PE per cycle.

For a given target body i with position (x_i, y_i) , each PE processes a source body j with position (x_j, y_j) and mass m_j . Within each PE, the relative displacement is computed as

$$\Delta x = x_j - x_i, \quad \Delta y = y_j - y_i$$

and the squared distance is formed as

$$r^2 = (\Delta x)^2 + (\Delta y)^2.$$

The squared distance is used to index the local lookup table, which produces the programmable radial interaction function $g(r^2)$. Each PE then computes a two-dimensional interaction contribution proportional to the displacement vector,

$$\Delta \mathbf{F}_{i \leftarrow j} = m_i m_j g(r^2) \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix},$$

and forwards the partial result to the next stage of the systolic array. Contributions from successive PEs are accumulated in a pipelined fashion, producing the total force vector for the target body after all source bodies have been processed.

The lookup table memory is replicated per PE to enable independent $g(r^2)$ evaluation at each stage and to sustain full throughput without memory contention.

7.1.4 Lookup Table Memory

Distance-dependent interaction functions are implemented using on-chip SRAM lookup tables indexed by squared distance r^2 . To enable reconfiguration across a range of interaction laws without redesigning the compute datapath, the LUT output provides the scalar function $g(r^2)$ used by the accelerator core.

To reduce memory footprint while maintaining adequate functional fidelity, the NBU LUT is stored in a **piecewise-linear** format. The squared-distance code $r^2_code[15:0]$ is split into an upper index field and a lower fractional field. The upper bits select a LUT segment, while the lower bits provide interpolation within the selected segment. Each LUT entry stores a packed pair consisting of a **base** value and a **slope** value. The output is computed by scaling the slope by the fractional field and adding it to the base, producing a first-order approximation of $g(r^2)$ over the segment:

$$g(r^2) \approx \text{base}[\text{idx}] + \text{slope}[\text{idx}] \cdot \text{frac}$$

This piecewise-linear representation increases approximation error relative to a full-resolution table, but significantly reduces SRAM requirements and supports efficient reprogramming of interaction laws.

To eliminate memory contention and sustain full throughput, the LUT SRAM is **replicated per processing element**, allowing each PE to perform an independent $g(r^2)$ lookup every cycle.

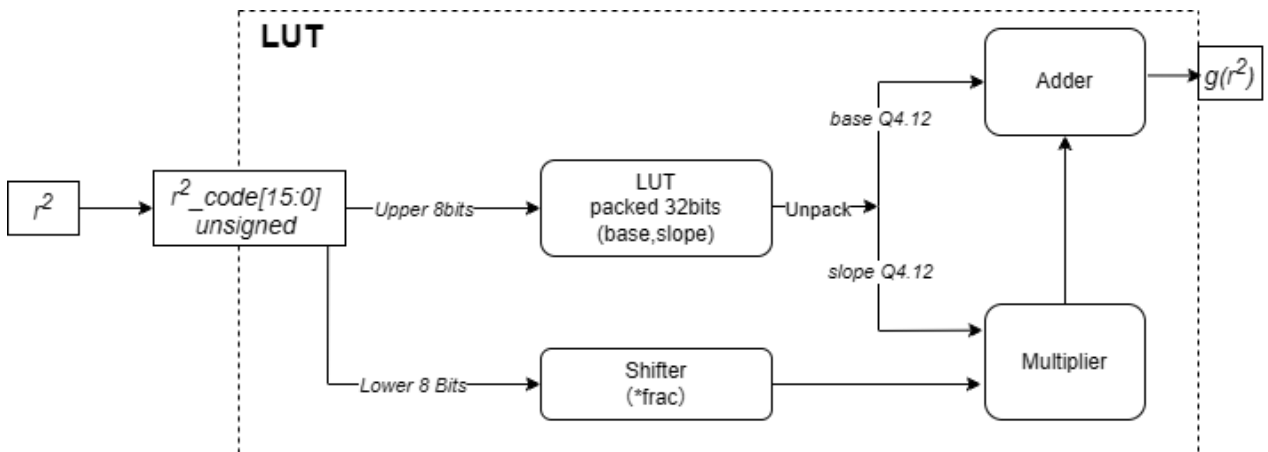


Figure 4: Piecewise-linear LUT structure for computing $g(r^2)$ using base-and-slope interpolation. The upper bits index the LUT segment while the lower bits scale the slope and are added to the base.

7.2 Finite State Machine (FSM)

The Stellavis NBU employs a hierarchical finite state machine to coordinate configuration, data movement, computation, and result output. A top-level FSM manages high-level operational phases, while two dedicated FSM handles systolic data setup and lookup-table configuration. This separation simplifies control flow and allows LUT configuration to be performed independently of computation.

7.2.1 Top Level FSM

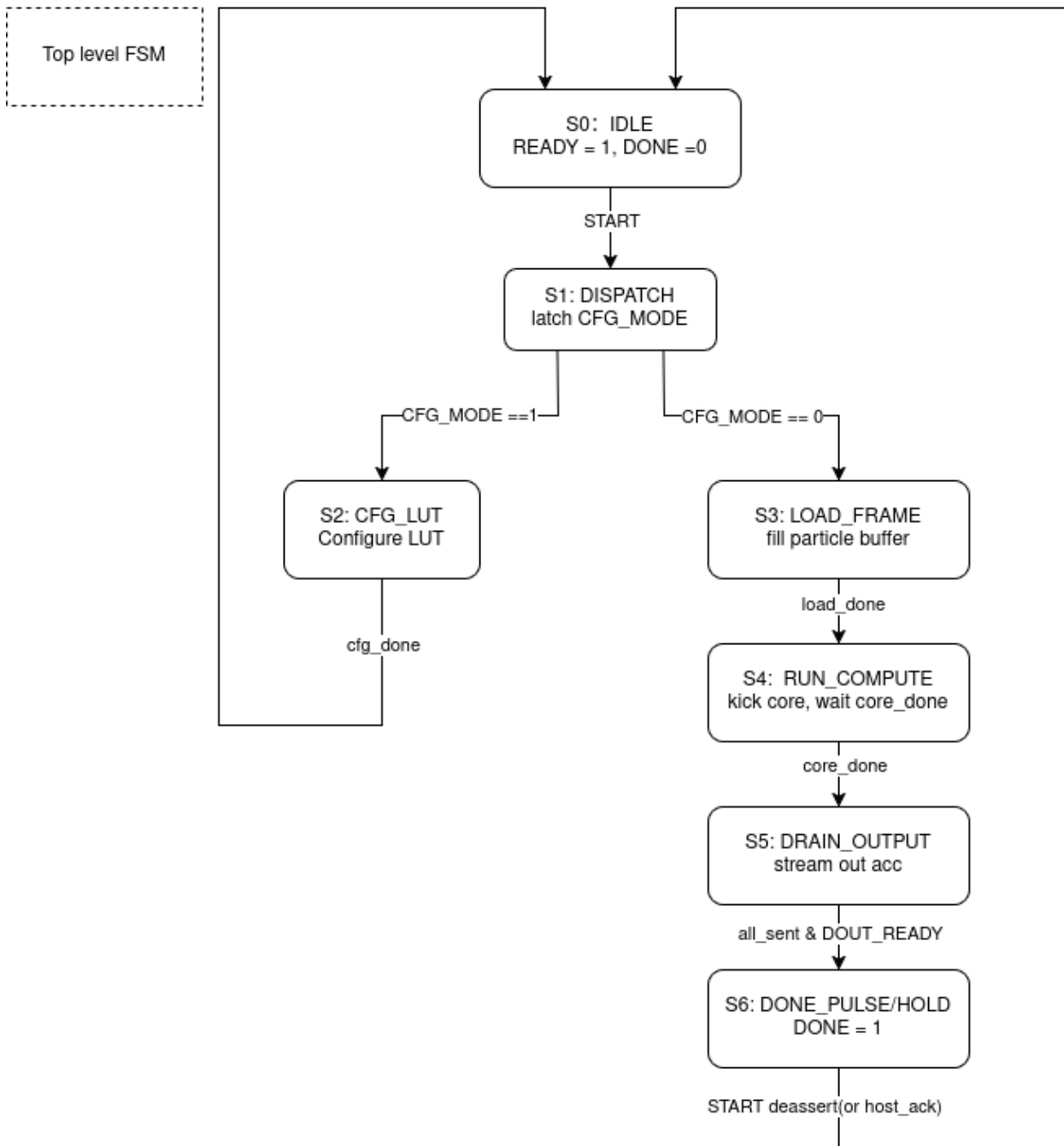


Figure 5: Top-level FSM Chart

The top-level FSM sequences the major operating phases of the NBU and interfaces with the host control logic. After reset, the device enters the **IDLE** state, where it asserts READY and waits for a START command from the host. Upon receiving START, the FSM transitions to the **DISPATCH** state, where the configuration mode is latched to determine whether LUT programming or computation is requested. If configuration mode is selected, the FSM enters the **CFG_LUT** state and transfers control to the Configure-LUT FSM. Upon completion of LUT programming, control returns to the IDLE state. If compute mode is selected, the FSM proceeds to **LOAD_FRAME**, where particle state data is streamed into the internal data buffers. Once all input data has been received, the FSM asserts control to the accelerator core in the **RUN_COMPUTE** state. During this state, the systolic processing element array evaluates pairwise interactions and accumulates per-body acceleration results. After computation completes, the FSM transitions to **DRAIN_OUTPUT**, streaming accumulated acceleration results to the output interface. When all output data has been successfully transmitted, the FSM enters the **DONE** state and asserts a completion signal. The FSM remains in this state until the host deasserts START, after which the device returns to IDLE.

State	Description
IDLE	Awaits host START command; READY asserted, DONE deasserted
DISPATCH	Latches configuration mode and selects configuration or compute flow
CFG_LUT	Invokes LUT configuration FSM and waits for completion
LOAD_FRAME	Streams particle data into internal buffers
RUN_COMPUTE	Activates accelerator core and waits for computation to complete
DRAIN_OUTPUT	Streams accumulated acceleration results to output interface
DONE	Signals completion to host and waits for START deassertion

Table 2: Top-Level FSM state summary.

7.2.2 Configure-LUT FSM

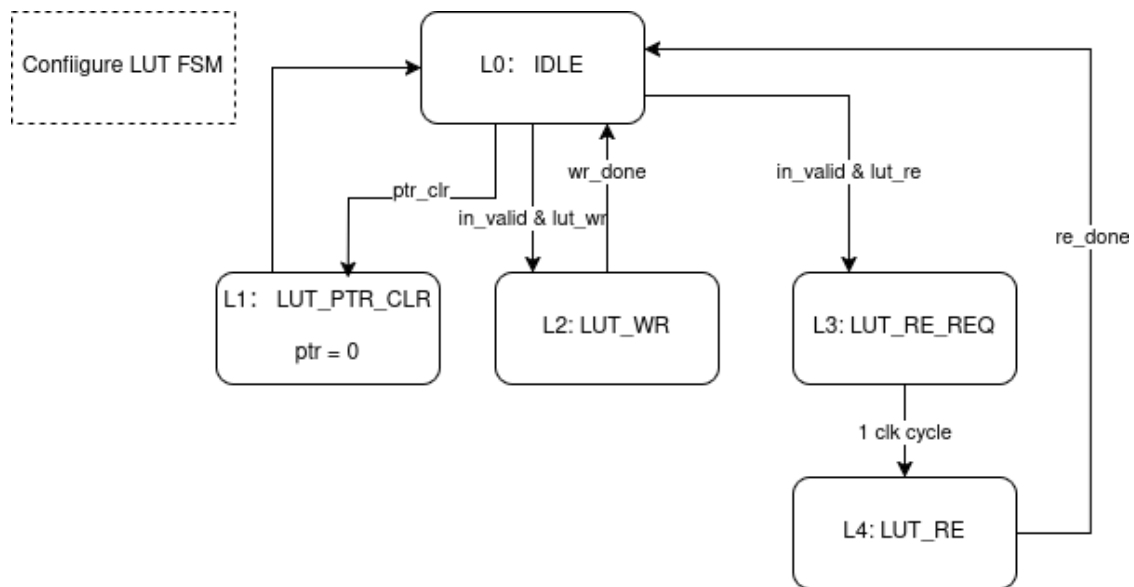


Figure 6: Configure LUT FSM Chart

The Configure-LUT FSM manages programming and optional verification of the distance-dependent interaction lookup tables. It operates independently of the accelerator core and is invoked by the top-level FSM when LUT configuration is requested.

The FSM begins in the **IDLE** state and awaits a configuration command. Upon entry, the LUT address pointer is cleared to ensure deterministic addressing. In the **LUT_WR** state, the FSM accepts streamed write data from the host and writes entries sequentially into the LUT SRAM. When readback is requested, the FSM issues a **LUT_RE_REQ**, followed by a **LUT_RE** cycle to return LUT contents to the host for verification. Once all requested write and read operations are complete, the FSM signals completion to the top-level FSM and returns to the IDLE state.

State	Description
IDLE	Waits for LUT configuration command
LUT_PTR_CLR	Clears LUT address pointer to starting address
LUT_WR	Accepts and writes LUT data from host
LUT_RE_REQ	Issues LUT read request
LUT_RE	Returns LUT read data to host

Table 3: Configure-LUT FSM state summary.

7.3 DFT

The Stellavis NBU integrates a scan-based DFT debug interface that provides visibility into internal state during bring-up and verification. When scan mode is enabled, selected internal tap registers capture intermediate values from the control sequencer, FIFO and buffer status, LUT addressing and coefficient outputs, and representative processing-element pipeline and accumulator checkpoints. The captured snapshot is shifted out serially through the scan output using the scan clock, enabling external test equipment to read intermediate data and localize functional issues without relying on normal runtime data interfaces.

Block	Data Observed via Scan	Debug Purpose
Control Unit (FSM)	FSM state, phase counters	Identify control flow issues
Input FIFO and Buffer	wr_ptr, rd_ptr, empty/full	Debug data ingress and flow
PE Cluster	Accumulator registers	Verify arithmetic progression
LUT	Address pointer, output registers	Validate LUT configuration and access
Output Logic	Output buffer, shift counters	Isolate output path errors

Table 4: Key scan tap points for internal debug visibility.

8 Application Information

8.1 FPGA-Based System Integration

The Stellavis NBU is designed to operate as a coprocessor tightly coupled to an external FPGA or microcontroller. A parallel data interface combined with simple handshaking and control signals enables deterministic, high-throughput streaming of particle data and force results, while a centralized control finite state machine sequences configuration and computation.

8.1.1 Parallel Data Interface

The NBU communicates with the FPGA using a 16-bit parallel data bus in both input and output directions. Data transfer is governed by ready/valid handshaking to decouple host timing from the internal accelerator pipeline.

- **Input path:** The FPGA drives `DIN[15:0]` along with `DIN_VALID`. The NBU asserts `DIN_READY` when it is able to accept new data. Input data represents particle parameters during compute mode or lookup-table data during configuration mode.
- **Output path:** The NBU drives `DOUT[15:0]` along with `DOUT_VALID`. The FPGA asserts `DOUT_READY` to acknowledge data reception. Output data consists of accumulated force components or LUT readback data depending on the operating mode.

This interface supports continuous streaming, allowing the FPGA to pipeline data transfers without stalling the accelerator core.

8.1.2 Control and Mode Selection

Operation of the NBU is controlled through a small set of synchronous control signals managed by the FPGA.

- CFG_MODE selects between lookup-table configuration mode and normal compute mode.
- START initiates the requested operation.
- READY indicates that the NBU is idle and prepared to accept a new command.
- DONE signals completion of the current operation.

The FPGA asserts START only when READY is high. Upon completion, the NBU asserts DONE and holds it until START is deasserted, providing a deterministic and robust command protocol.

8.1.3 Lookup-Table Configuration Protocol

In lookup-table configuration mode, the FPGA programs the distance-dependent interaction function by streaming LUT data through the input interface.

1. Assert CFG_MODE = 1.
2. Clear the LUT address pointer using LUT_PTR_CLR.
3. Stream LUT entries on DIN[15:0] while asserting LUT_WR.
4. Optionally request LUT readback using LUT_RD and monitor LUT_RVALID.
5. Deassert CFG_MODE and return to idle.

8.1.4 Compute Operation Protocol

In normal compute mode, the FPGA streams particle data frames into the NBU and receives computed force results.

1. Assert CFG_MODE = 0.
2. Stream particle positions and masses using the input data interface.
3. Assert START to begin computation.
4. Monitor DONE to detect completion.
5. Stream force results from the output data interface.

FIFO status signals such as input and output full or empty flags may be monitored for debugging or flow-control verification during system bring-up.

8.2 Functional Applications and Supported Physics Laws

The Stellavis NBU accelerates pairwise interaction evaluation for systems governed by central, distance-dependent force laws. The accelerator computes forces of the form

$$\mathbf{F}_{i \leftarrow j} = k_{ij} g(r^2) \begin{bmatrix} x_j - x_i \\ y_j - y_i \end{bmatrix}, \quad r^2 = (x_j - x_i)^2 + (y_j - y_i)^2,$$

where the scalar function $g(r^2)$ is provided by the programmable lookup table and the coefficient k_{ij} encodes per-body parameters such as mass or charge.

8.2.1 Newtonian Gravity

Newtonian gravity models long-range attractive interaction between masses and is widely used in astrophysical and orbital simulations.

$$\mathbf{F}_{i \leftarrow j} = G m_i m_j \frac{\mathbf{r}}{r^3}.$$

This law is implemented by programming the LUT to approximate

$$g(r^2) = \frac{G}{(r^2)^{3/2}},$$

with optional softening for numerical stability.

8.2.2 Coulomb Electrostatic Interaction

Electrostatic interaction models attraction or repulsion between charged particles and shares the same inverse-square spatial dependence as gravity.

$$\mathbf{F}_{i \leftarrow j} = k_e q_i q_j \frac{\mathbf{r}}{r^3}.$$

This interaction is implemented using the same LUT shape as gravity, with per-body coefficients encoding charge magnitude and sign.

8.2.3 Lennard–Jones 12–6 Interaction

The Lennard–Jones force models short-range repulsion and moderate-range attraction and is commonly used in molecular dynamics.

$$\mathbf{F}_{i \leftarrow j} = 24\epsilon \left(2 \frac{\sigma^{12}}{r^{14}} - \frac{\sigma^6}{r^8} \right) \mathbf{r}.$$

This nonlinear interaction is realized by programming the LUT to approximate the corresponding $g(r^2)$ profile using the piecewise-linear base-and-slope representation.

8.2.4 Linear Spring Interaction

The linear spring interaction models elastic coupling between particles, as if connected by ideal springs with zero rest length.

$$\mathbf{F}_{i \leftarrow j} = k \mathbf{r}.$$

This interaction corresponds to a constant LUT output $g(r^2) = k$ and demonstrates the ability of the NBU to implement simple linear force laws with minimal memory overhead.

8.2.5 Application Flexibility

By combining a programmable $g(r^2)$ lookup table with a high-throughput systolic compute core, the Stellavis NBU supports a wide range of interaction laws without modifying the underlying hardware. This architecture enables rapid experimentation across physical models and efficient FPGA-based system integration.